# OPERATING SYSTEMS

UE25CS242A

PES UNIVERSITY

CELEBRATING 50 YEARS

## File Management

**Pavan A C**

Department of Computer Science & Engineering

# OPERATING SYSTEMS

## File Management – Free space management, Efficiency and Performance

**Pavan A C**
Department of Computer Science & Engineering

# OPERATING SYSTEMS

## Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

PES UNIVERSITY
CELEBRATING 50 YEARS

# Free-Space Management




- Need to reuse the disk space from deleted files for new files
- To keep track of free disk space, the system maintains a **free-space list**
- The free-space list records all **free disk blocks**
- When a file is created, the free-space list is searched for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list.
- When a file is deleted, its disk space is added to the free-space list

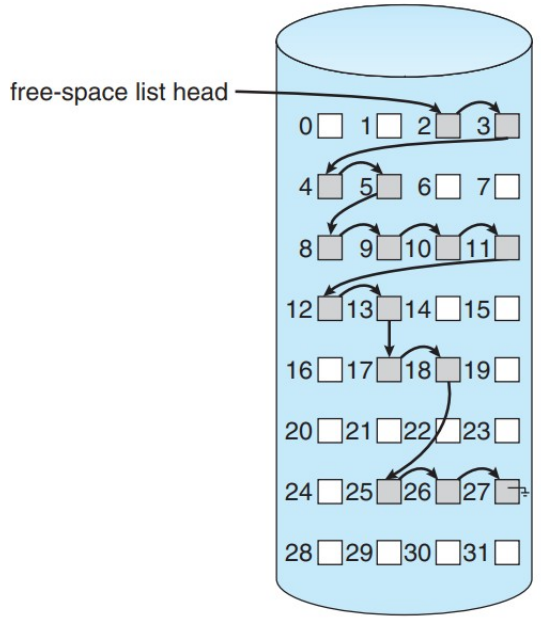
## Bit Vector




- The free-space list is implemented as a **bit map** or **bit vector**.
- Each block is represented by 1 bit.
- If the block is free, the bit is 1; if the block is allocated, the bit is 0
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The free-space bit map would be 00111100111111000110000011100000 …
- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk

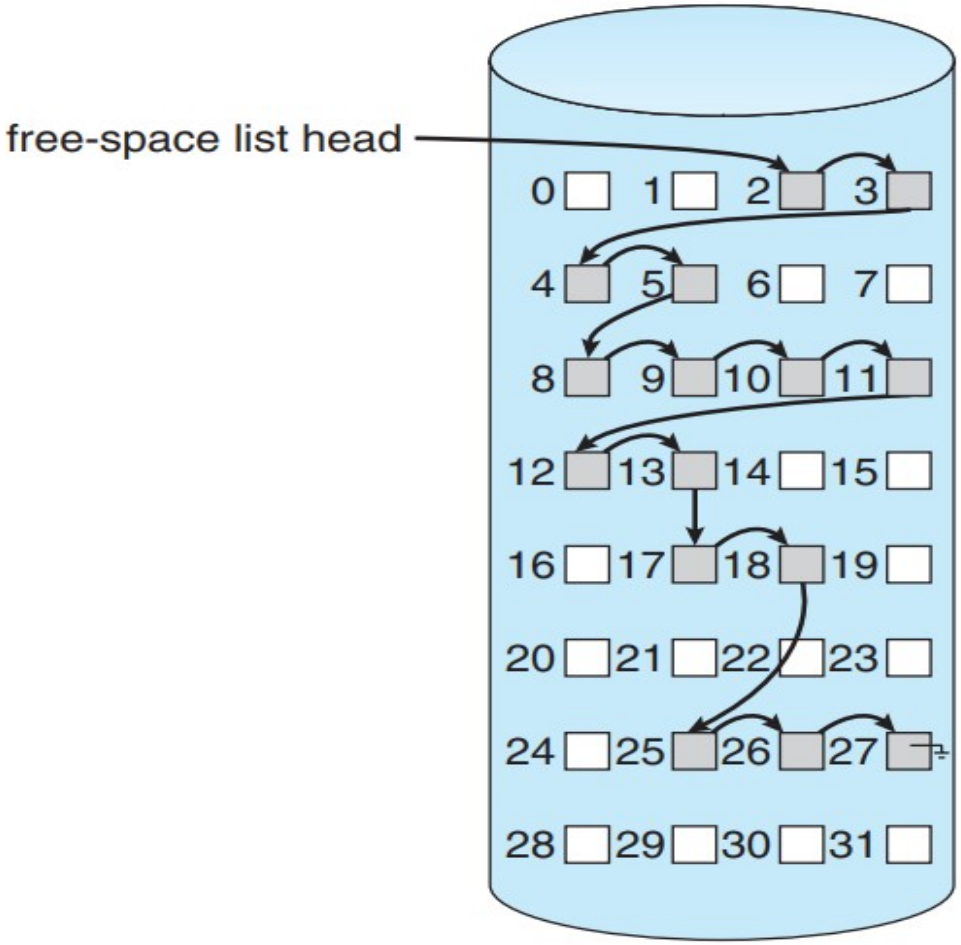
# Linked List

- Link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- The first free block contains a pointer to the next free disk block, and so on
- **Example:** Consider blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The pointer would point to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4 and so on
- This scheme is not efficient; to traverse the list, one must read each block, which requires substantial I/O time.



# OPERATING SYSTEMS

## Linked List



## Grouping




- A modification of the free-list approach stores the addresses of n free blocks in the first free block.
- The first n−1 of these blocks are actually free.
- The last block contains the addresses of another n free blocks, and so on.
- The addresses of a large number of free blocks can now be found quickly, unlike the situation when the standard linked-list approach is used.

## Counting

PES UNIVERSITY
CELEBRATING 50 YEARS

- Generally, several contiguous blocks may be allocated or freed simultaneously, particularly when space is allocated with the contiguous-allocation algorithm or through clustering.
- Rather than keeping a list of n free disk addresses, we can keep the address of the first free block and the number (n) of free contiguous blocks that follow the first block
- Each entry in the free-space list then consists of a disk address and a count.
- The entries can be stored in a balanced tree, rather than a linked list, for efficient lookup, insertion, and deletion.

## Efficiency and Performance




Efficiency dependent on:

- Disk allocation and directory algorithms
- Types of data kept in file's directory entry
- Pre-allocation or as-needed allocation of metadata structures
- Fixed-size or varying-size data structures

Performance dependent on:

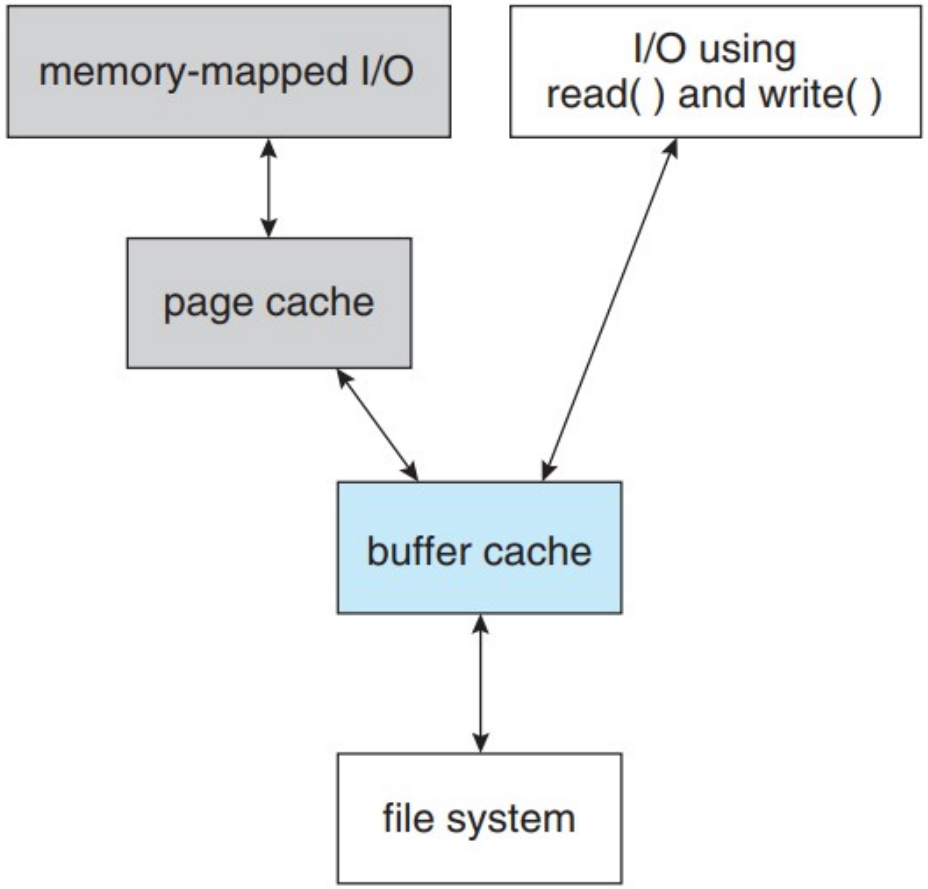
- Keeping data and metadata close together
- Buffer cache – separate section of main memory for frequently used blocks
- Free-behind and read-ahead – techniques to optimize sequential access
- Reads frequently slower than writes

# Page cache

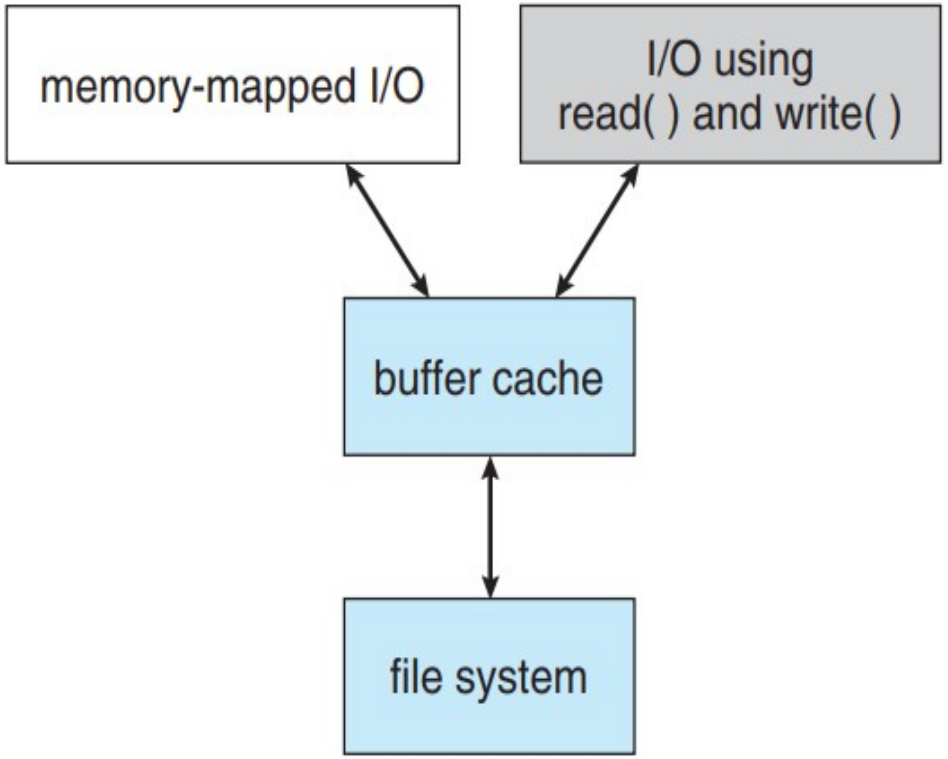
PES UNIVERSITY
CELEBRATING 50 YEARS

- Some systems maintain a separate section of main memory for a **buffer cache**, where blocks are kept under the assumption that they will be used again shortly.
- Other systems **cache file data using a page cache**. The page cache uses virtual memory techniques to cache file data as pages rather than as file-system-oriented blocks.
- Caching file data using virtual addresses is far more efficient than caching through physical disk blocks, as accesses interface with virtual memory rather than the file system.
- Several systems—including Solaris, Linux, and Windows —use page caching to cache both process pages and file data. This is known as **unified virtual memory**.

# I/O without a unified buffer cache



# I/O using a unified buffer cache



# Page replacement




- Sequential access can be optimized by techniques known as free-behind and read-ahead.
- **Free-behind** removes a page from the buffer as soon as the next page is requested. The previous pages are not likely to be used again and waste buffer space.
- With **read-ahead**, a requested page and several subsequent pages are read and cached. These pages are likely to be requested after the current page is processed.
- Retrieving these data from the disk in one transfer and caching them saves a considerable amount of time




# THANK YOU

**Pavan A C**

Department of Computer Science and Engineering

**pavanac@pes.edu**